

ArogyaAI ML and RAG Architecture Report

Model Strategy and Architecture Evaluation

Pre-training phase | Production intelligence layer recommendation

Generated for the ArogyaAI system: FastAPI backend, ML pipeline, RAG pipeline, wearable ingestion, and SHAP explainability.

Scope: read-only analysis of the current repository and practical recommendations for the future production architecture.

Workspace evidence date: 2026-05-01 IST. No existing code files were modified to produce this report.

Decision Area	Recommended Direction
Best ML model	Calibrated LightGBM/XGBoost gradient-boosted tabular model with disease-specific heads, SHAP, monotonic constraints where clinically appropriate, and RandomForest retained as a baseline.
Best RAG architecture	Qdrant hybrid retrieval using dense + sparse vectors, RRF fusion, medical metadata filters, reranking, evidence-gated generation, and source/audit logging.
Recommended embeddings	Self-hosted BGE-M3 for privacy-preserving hybrid retrieval; use OpenAI text-embedding-3-large only if PHI governance and cost controls allow.
Recommended LLM	Do not use Ollama Llama 3.1 8B as the only production clinical assistant. Use it for local/private fallback; use a stronger hosted or larger self-hosted model for production chat and high-risk cases.

Table Of Contents

1. Current System Analysis
 2. ML Model Comparison
 3. RAG Architecture Comparison
 4. Best Architecture Recommendation
 5. Training Strategy
 6. RAG Improvement Plan
 7. Roadmap
 8. Risks
 9. Final Verdict
- Appendix A. Repository Evidence
- Appendix B. Evaluation Checklists
- Appendix C. References

Executive Summary

ArogyaAI already has a strong application and data foundation for a health intelligence product. The backend persists feature snapshots, risk scores, SHAP values, health scores, labs, vitals, reports, and clinical history. Google Fit ingestion, lab parsing, onboarding data, and timeline aggregation give the system the right raw materials for personalized risk intelligence.

The current intelligence layer, however, is still a prototype. The deployed ML artifact is a RandomForestClassifier trained on a synthetic bootstrap dataset of 1,200 rows with six features: bmi, hr_mean_7d, steps_avg_7d, sleep_efficiency, lifestyle_score, and activity_score. The pipeline collects richer signals, including age, sex, blood pressure, glucose, cholesterol, family history, disease flags, symptom flags, severity, and clinical history, but these are not yet used by the deployed model.

The RAG system is structurally promising but data-poor. It uses Qdrant, fastembed, BAAI/bge-small-en-v1.5 embeddings, paragraph chunking, top-k retrieval, and Ollama/OpenAI-compatible generation fallback. The actual medical corpus is only four Markdown files, about 1,300 total words. That is enough for a demonstration of grounded explanations, but not enough for production medical coverage.

The best near-term production path is not a neural time-series model. ArogyaAI should first move to a calibrated gradient-boosted tree model, preferably LightGBM or XGBoost, trained on real disease labels and richer feature snapshots. This model class fits ArogyaAI's current data shape: tabular, mixed-quality, partially missing, clinically structured, and explainability-sensitive. Time-series neural models should be deferred until the product has longitudinal outcomes, enough users, stable wearable streams, and a validated temporal labeling strategy.

For RAG, the best production architecture is hybrid retrieval in Qdrant: dense vectors plus sparse lexical retrieval, RRF fusion, metadata filters, reranking, evidence grading, and source-grounded answer generation. BGE-M3 is the most aligned self-hosted embedding recommendation because it supports dense, sparse, and multi-vector retrieval modes, larger context than BGE v1.5, and multilingual future expansion. OpenAI embeddings are a strong option only if ArogyaAI can satisfy PHI, consent, residency, and cost governance.

The final recommendation is conservative by design: upgrade the model and RAG layer in phases, prove calibration and safety before deeper automation, and keep medical outputs framed as decision support rather than diagnosis.

Critical Verdict

Best ML model for ArogyaAI: calibrated LightGBM/XGBoost gradient-boosted tabular risk model with disease-specific heads and SHAP.

Best RAG architecture: Qdrant hybrid dense+sparse retrieval with BGE-M3, RRF fusion, reranking, curated medical corpus, evidence-gated generation, and safety escalation.

Analysis Method

This report was produced by inspecting the local repository and comparing the observed architecture against current production ML/RAG patterns. The analysis was read-only except for generating this PDF artifact.

Area	Repository Evidence
ML model	pipelines/ml_pipeline/train.py, preprocess.py, model_loader.py, inference.py, service.py, shap_explainer.py; artifact apps/backend/models/diabetes.pkl
Features	pipelines/feature_pipeline/service.py and aggregation.py; apps/backend/models/feature_snapshot.py
Data sources	apps/backend/services/google_fit_service.py, lab_pipeline_service.py, clinical_history_service.py; timeline route; user_profile, user_vital, wearable_data, lab_result, report models
RAG	pipelines/rag_pipeline/config.py, corpus.py, embedder.py, retriever.py, query_builder.py, generator.py, service.py; data/medical_corpus
Chat integration	apps/backend/services/chat_service.py and routes/chat.py; frontend Chatbot component
Persistence and orchestration	pipelines/storage_pipeline/service.py, pipelines/orchestrator.py, pipelines/orchestration_pipeline/tasks.py, docker-compose.yml

The external technical references used in the recommendation are listed in Appendix C. They include official documentation for scikit-learn RandomForestClassifier, XGBoost, LightGBM, SHAP TreeExplainer, Qdrant hybrid queries, BGE models, OpenAI embeddings, Ollama Llama 3.1, FDA transparency principles, TRIPOD+AI, and PROBAST+AI.

1. Current System Analysis

Current ML Pipeline

The current ML pipeline is implemented as an application-integrated pipeline, not only a standalone microservice. The production-facing path builds a feature snapshot, loads a local model artifact, runs probability inference, stores risk and health score records, persists SHAP factors, and then hydrates the prediction with RAG explanations.

Component	Observed Current State	Production Interpretation
Model artifact	apps/backend/models/diabetes.pkl contains a dict with model, feature_names, model_version, label, positive_class_index, and training_summary.	Good artifact envelope, but missing model card, validation report, data lineage, calibration metadata, and approval status.
Model type	RandomForestClassifier with 250 trees, max_depth=8, min_samples_leaf=4.	Reasonable prototype baseline. Not the best final choice for calibrated clinical risk on structured data.
Training data	Synthetic bootstrap dataset of 1,200 rows with positive_rate about 0.554.	Not acceptable for production disease prediction. Useful only for plumbing, UI, and integration testing.
Target	label is diabetes_risk.	ArogyaAI needs explicit disease-specific targets, outcome definitions, time horizons, and label provenance.
Deployed features	bmi, hr_mean_7d, steps_avg_7d, sleep_efficiency, lifestyle_score, activity_score.	A narrow feature set leaves major collected clinical signals unused.
Inference	predict_proba returns positive class probability; risk levels use 0.25, 0.50, 0.75 cutoffs.	Fast and simple, but cutoffs are not clinically calibrated.
Explainability	SHAP TreeExplainer is cached by model path and SHAP rows are stored in DB.	Strong explainability mechanism for tree models; explanation validity depends entirely on training data validity.

Current Data Sources

The system's data foundation is broader than the deployed model. FeaturePipelineService pulls from profile/onboarding, clinical history, medical history, lab results, vitals, Google Fit, wearable_data, and sleep summaries. It creates both direct values and proxy values such as HRV proxy, resting heart-rate proxy, cholesterol proxy, sleep score, activity score, lifestyle score, and data completeness.

Data Source	Current Usage	Important Gap
Wearables	Google Fit sync persists UserVital rows for steps, heart rate, sleep, SpO2, and legacy WearableData daily steps. Feature aggregation uses 7-day and 30-day windows.	No robust longitudinal feature store yet for trend slopes, variability, adherence, missingness, and circadian patterns.
Lab reports	Regex extraction normalizes hemoglobin, WBC, glucose, HbA1c, creatinine, lipid markers, TSH, etc. LabResult rows are available for feature snapshots.	Only glucose and cholesterol are partially consumed by feature snapshots; deployed model does not use them.
Baseline onboarding	UserProfile stores age, gender, height, weight, activity, goals, family history, medications, sleep, stress, smoking, alcohol, allergies.	Profile features are collected but mostly absent from deployed RF feature vector.
Clinical history	Structured complaint, duration, onset, severity, associated symptoms, negative symptoms, aggravating/relieving factors.	Clinical features are used for enrichment and chat, not yet modeled as supervised predictors.
Timeline/history	Timeline route joins vitals, labs, alerts, reports, and clinical history into user-facing history.	Useful for RAG and temporal summarization; not converted into validated temporal ML features yet.

Current RAG Pipeline

The RAG pipeline has a clean modular structure: settings, corpus loading, embedding, Qdrant retrieval, SHAP-query building, generator, and explanation service. Its default embedding model is BAAI/bge-small-en-v1.5 with 384 dimensions. The Qdrant collection is medical_knowledge. Retrieval top_k defaults to 4, and chunks are built from Markdown paragraphs with 300-500 word targets.

The present corpus is very small: four files covering activity/cardiovascular risk, BMI impact, diabetes risk factors, and sleep/metabolic health. Each file is about 300-350 words, so the current index effectively contains about four topical chunks. This means retrieval quality is currently limited by corpus coverage, not by Qdrant itself.

RAG Attribute	Current Implementation	Assessment
Vector DB	Qdrant service in docker-compose, collection medical_knowledge.	Good production-compatible base.
Embedding model	BAAI/bge-small-en-v1.5 via fastembed, 384 dimensions.	Fast and local; limited recall compared with larger or hybrid-capable models.
Chunking	Paragraph merge to 300-500 words.	Reasonable for small Markdown notes; insufficient for long guidelines without section-aware metadata.
Retrieval	Dense vector search only in core pipeline; chat has lexical fallback if vector retrieval fails.	No hybrid retrieval, no reranker, no retrieval evaluation.
Generation	Ollama /api/generate with JSON format if OLLAMA_BASE_URL is configured; OpenAI-compatible chat completion fallback if API base/key are configured; otherwise deterministic fallback response.	Practical resilience; 8-second timeout may be too strict for local LLMs and too loose for urgent chat fallback if not cached.
Grounding	Prompt says use only retrieved context and return JSON.	Good start, but no post-generation citation verifier or unsupported-claim detector.

Chat Integration

The chat service combines three streams before answering: latest ML prediction/explanation, user health context, and retrieved medical context. It retrieves latest vitals, labs, reports, feature snapshot, clinical history, ML risk scores, SHAP drivers, recommendations, and RAG documents. It then asks a JSON-only clinical assistant prompt to respond cautiously, avoid final diagnosis, ask follow-up questions, and include safety notes.

- Strength: the assistant is already personalized. It does not answer only from generic RAG; it combines ML risk, vitals, labs, history, timeline, symptoms, and retrieved corpus.
- Strength: emergency patterns and high-risk symptoms are explicitly escalated in fallback logic.
- Weakness: if no LLM is configured, the fallback can be safe but generic. If an LLM is configured, there is no strong hallucination audit after generation.
- Weakness: chat may trigger prediction refresh if no latest risk exists, which can add latency and create a user-visible pause.
- Weakness: conversation history is capped at three messages. That is good for latency but limits longitudinal conversational reasoning.

Strengths

- Strong persistence model: feature_snapshots, risk_scores, shap_values, health_scores, lab_results, user_vitals, reports, clinical_history, and timeline data are persisted with user-level linkage.
- End-to-end prediction flow exists: feature snapshot -> model inference -> SHAP -> storage -> RAG explanation -> notification -> UI.
- Tree SHAP is already integrated and compatible with the recommended gradient-boosted tree path.
- Celery orchestration and Redis are available for background feature, inference, SHAP, and baseline computation.
- Qdrant is already in docker-compose, so the system can evolve from dense-only retrieval to hybrid retrieval without changing the whole platform architecture.
- The chat layer already applies cautious medical wording and red-flag fallback behavior.

Weaknesses And Bottlenecks

- The current ML model is trained on synthetic data. It should not define real disease risk decisions.
- The deployed RF feature vector uses only six features, while the application collects many clinically relevant signals.
- Missing values are often coerced to 0.0 for model features. This can create false signals unless missingness is modeled explicitly.

- No repository evidence was found for train/validation/test splits, AUC, PR-AUC, calibration, subgroup evaluation, confidence intervals, or drift monitoring.
- Risk thresholds are product thresholds, not calibrated clinical thresholds.
- RAG corpus coverage is far too small for production medical answering.
- Current RAG does not use hybrid dense+sparse retrieval, reranking, guideline provenance, evidence grade, or medical claim verification.
- LLM selection is configuration-driven but not governed by safety tier, data sensitivity, or latency SLA.

2. ML Model Comparison

The model choice should match ArogyaAI's pre-training phase. The immediate problem is supervised tabular disease risk from heterogeneous health data, not raw waveform modeling. Most data currently lands as structured rows and aggregates: profile, labs, vitals windows, feature snapshots, and clinical flags. For that shape, gradient-boosted decision trees are the strongest practical production step.

Model	Accuracy Potential	Explainability	Training Complexity	Inference Speed	Healthcare Suitability
RandomForest (current)	Moderate. Stronger than a single tree but often below tuned GBDT on tabular risk tasks. Current artifact is synthetic, so actual accuracy is unknown.	Good with Tree SHAP and permutation importance. Can be overconfident and impurity feature importances can mislead.	Low. Easy to train and deploy. Robust baseline.	Fast enough for online inference. SHAP acceptable with six features.	Good baseline, not final production model. Keep as benchmark and fallback only.
XGBoost / LightGBM	High for structured tabular data with nonlinear interactions, missingness, and mixed feature distributions.	Strong with Tree SHAP. Supports feature constraints, calibration, and clinically readable driver explanations.	Medium. Requires data validation, hyperparameter tuning, calibration, and drift checks.	Very fast online. Batch SHAP can be precomputed or cached.	Best near-term fit for ArogyaAI. Production recommendation.
Neural Network (MLP)	Medium to high only with more data and careful regularization. Can underperform GBDT on smaller tabular datasets.	Weaker. SHAP/Integrated Gradients possible but less intuitive and more expensive.	Medium to high. Needs scaling, architecture tuning, class imbalance strategy, and overfitting controls.	Fast inference, but explanation overhead higher.	Not recommended as primary now. Consider later for representation learning or multimodal fusion.
Time-series LSTM / Transformer	Potentially high for raw longitudinal wearable streams if there are many labeled sequences and stable sampling.	Weak to moderate. Attention is not a full explanation; sequence SHAP is expensive.	High. Needs temporal labels, sequence alignment, missingness handling, leakage control, and large samples.	Moderate to slow depending on sequence length. More infrastructure.	Not phase-1. Use only after validated longitudinal labels and enough users.
Hybrid tabular + time-series	Highest long-term ceiling: combines static risk factors, labs, trends, and raw sequence dynamics.	Mixed. Tabular head can use SHAP; temporal module needs separate explanation strategy.	High. Requires model registry, feature store, temporal validation, and monitoring.	Can be optimized with cached embeddings/features.	Best long-term architecture, not first production step.

Why RandomForest Should Not Remain The Final Model

- The current RandomForest is trained from a synthetic formula, so its confidence and SHAP values reflect that formula rather than real patient outcomes.
- The model omits clinically relevant features already present in the feature snapshot and database, including glucose, cholesterol, age, sex, blood pressure, family history, symptoms, and disease flags.
- RandomForest probabilities are often poorly calibrated without explicit calibration. In healthcare, a risk score should be treated as a probability only after calibration is measured.
- It is useful as a continuity baseline because it is integrated, fast, and compatible with existing SHAP storage.

Why Gradient-Boosted Trees Are The Best Next Step

- They fit ArogyaAI's data shape: tabular, mixed provenance, missing values, aggregates, flags, and nonlinear feature interactions.
- They are explainable with the same SHAP family already used by the repository.
- They can be calibrated and evaluated with clinically meaningful metrics.
- They allow disease-specific heads: diabetes, cardiovascular, respiratory, sleep/metabolic, and general deterioration risk.
- They are fast enough for synchronous API predictions and can run inside the current FastAPI/Celery stack.
- They create a reliable baseline before expensive sequence models are introduced.

Disease-Specific Modeling Recommendation

ArogyaAI should not train one generic 'health risk' model and ask it to explain every disease. The system should train disease-specific heads with explicit time horizons and label definitions. A shared feature store can serve several supervised heads.

Target Head	Suggested Label	Useful Features	Notes
Diabetes / metabolic risk	New diagnosis, HbA1c/glucose threshold crossing, clinician-confirmed risk class, or validated lab-based proxy for pre-training.	BMI, age, sex, family history, glucose, HbA1c, activity, sleep, triglycerides, HDL/LDL, BP, medications.	Most aligned with current artifact but requires real labels.
Cardiovascular risk	Hypertension diagnosis, high BP persistence, ASCVD-like risk category, cardiac event proxy, or clinician-labeled risk.	BP, resting HR, HRV proxy, BMI, lipids, smoking, activity, sleep, age, family history.	Use monotonic constraints for BP, age, smoking where validated.
Sleep/recovery risk	Persistent poor sleep or recovery state, symptom escalation, fatigue/insomnia labels.	Sleep duration, sleep efficiency, HRV/RHR, stress, activity, symptoms.	May start as a rules+GBDT hybrid until labels mature.
Respiratory/environmental risk	Symptoms plus SpO2 or AQI-linked exacerbation labels if available.	SpO2, respiratory symptoms, pollution/AQI, sleep, heart rate, history.	Needs external environmental data and careful confounding controls.

Recommended ML Decision Matrix

Criterion	Current RF	LightGBM/XGBoost	MLP	LSTM/Transformer	Hybrid Long-Term
Accuracy	2/5	4/5 near term	3/5 until large data	2/5 now, 4/5 later	5/5 later
Calibration	2/5 without calibration	4/5 with isotonic/Platt	3/5	3/5	4/5
Explainability	4/5	4/5	2/5	2/5	3/5
Implementation fit	5/5 already built	4/5 modest change	3/5	2/5	2/5
Healthcare readiness	2/5 due synthetic data	4/5 after validation	2/5	2/5 now	4/5 with mature data
Inference cost	Low	Low	Low	Medium	Medium

3. RAG Architecture Comparison

The current RAG stack has the right bones but not enough retrieval quality controls for production medical use. The recommendation is to keep Qdrant, replace dense-only retrieval with hybrid retrieval, use a richer embedding/reranking strategy, expand the corpus, and add hallucination-reduction controls.

Architecture	Retrieval Accuracy	Hallucination Reduction	Latency	Cost	Scalability	Assessment
Current: Qdrant + BGE-small	Low to medium. Fast dense retrieval, but corpus and model are small.	Medium in prompt, low in verification. Fallback is safe but generic.	Low. 384-dim vectors and top_k=4 are fast.	Low and self-hosted.	Good for prototype.	Keep as baseline. Not production medical coverage.
BGE-large / Instructor	Medium to high for English semantic retrieval. Larger vectors improve recall at higher storage/latency cost.	Better recall can reduce unsupported answers, but still needs reranking and source checks.	Medium.	Low to medium self-hosted depending hardware.	Good.	BGE-large is a direct dense upgrade; Instructor can help domain prompts but is older and heavier.
OpenAI embeddings	High, especially text-embedding-3-large; text-embedding-3-small is strong cost option.	Better semantic recall, but still requires source validation and PHI governance.	Low to medium network latency.	Variable API cost; higher for large.	Excellent managed scalability.	Use only if PHI, consent, residency, and vendor agreements are acceptable.
Hybrid BM25 + vector	High. Captures exact medical terms, biomarkers, abbreviations, and semantic paraphrases.	High when paired with citations and thresholds.	Medium. Two retrieval branches plus fusion.	Low to medium.	Strong in Qdrant with named vectors/sparse vectors.	Best production default.
Multi-hop retrieval	High for complex questions involving labs + symptoms + disease mechanisms.	High if each hop is evidence-gated.	Medium to high.	Medium.	Requires orchestration and eval.	Use selectively for complex clinical chat, not every query.

Why Hybrid Retrieval Is Needed

- Medical queries often include exact terms, abbreviations, lab names, and thresholds. Dense embeddings can miss exact lexical matches.
- Users also ask vague natural-language questions. Sparse/BM25 alone can miss semantic intent.
- Hybrid dense+sparse retrieval handles both modes and lets Qdrant fuse results using RRF or related methods.
- Hybrid retrieval is especially useful for small-to-medium curated medical corpora where exact source sections matter.
- A reranker should then rank the top candidates for answerability, source quality, and patient-specific relevance.

Embedding Recommendation

For self-hosted production, BGE-M3 is the best aligned embedding model because it supports multi-functionality: dense retrieval, sparse lexical retrieval, and multi-vector retrieval. It also supports longer input lengths than BGE v1.5 and more than 100 languages. For ArogyaAI, that matters because medical users may mix English, local language terms, abbreviations, and patient-facing phrasing.

If cloud embeddings are permitted, OpenAI text-embedding-3-large is a high-quality option and text-embedding-3-small is a cost-efficient option. OpenAI's current embedding guide lists default vector lengths of 1536 for text-embedding-3-small and 3072 for text-embedding-3-large, with a dimensions parameter available to shorten embeddings. This should be considered only after PHI governance is settled.

LLM Recommendation

Ollama Llama 3.1 8B is useful for local development, privacy-sensitive demos, and deterministic fallback workflows. It should not be the sole production model for clinical chat because small local LLMs are more likely to miss nuance, produce

brittle JSON, or overstate evidence. A production system should use either a stronger hosted model under a compliant data-processing arrangement or a larger self-hosted model with GPU capacity and a formal evaluation harness.

LLM Option	Best Use	Risk
Ollama Llama 3.1 8B	Local development, privacy-preserving fallback, low-risk education responses, offline resilience.	Insufficient as sole production clinical assistant unless heavily evaluated and constrained.
Larger self-hosted LLM	Production privacy path when PHI cannot leave infrastructure.	High compute cost and ops complexity.
Hosted OpenAI-compatible model	Primary production chat if legal/PHI controls allow; better JSON reliability, reasoning, latency, and safety tooling.	Vendor governance, cost, data residency, and outage dependency.
Tiered model router	Recommended. Use deterministic fallback for urgent safety notes, cheaper model for routine education, stronger model for high-risk or complex questions.	Requires routing policy and monitoring.

Latency Targets

Path	Recommended Target	How To Achieve
Cached risk explanation	p50 < 500 ms, p95 < 1.5 s	Store RAG explanation cache keyed by prediction, SHAP, and clinical history. Current service already uses a cache key.
Fresh ML prediction	p50 < 1 s, p95 < 3 s	Feature snapshot queries optimized; model loaded once; SHAP cached or asynchronous for non-blocking views.
Routine chat	p50 < 4 s, p95 < 10 s	Hybrid retrieval < 300 ms, reranker < 700 ms, LLM streaming or fast JSON model, prompt trimmed to needed context.
Emergency / red flag response	Immediate fallback < 1 s	Do not wait for LLM. If emergency pattern matched, return safety note plus concise follow-up.
Corpus indexing	Offline/background	Do not index corpus on first user request. Current ensure_corpus_indexed should be moved to startup/job in production.

4. Best Architecture Recommendation

Best ML model for ArogyaAI: calibrated LightGBM/XGBoost gradient-boosted tabular model with disease-specific heads, SHAP explainability, calibration, and temporal feature engineering.

Best RAG setup: Qdrant hybrid dense+sparse retrieval with BGE-M3, RRF fusion, medical metadata filters, reranker, evidence-gated generation, and audited source citations.

Recommended Production Architecture

ArogyaAI Production Intelligence Layer

User data sources

Google Fit / wearables -> UserVital + WearableData
Lab report uploads -> Report + LabResult
Onboarding/profile -> UserProfile + MedicalHistory
Clinical intake -> ClinicalHistory
Timeline/history -> Timeline events + prior risk/SHAP records

|
v

Feature Store / Feature Snapshot Layer

- raw latest values
- 7d/14d/30d windows
- deltas, slopes, variability, missingness flags
- lab abnormality flags and clinical-history features
- source_breakdown and data_completeness

|
v

ML Risk Layer

- calibrated LightGBM/XGBoost disease heads
- diabetes, cardiovascular, sleep/metabolic, respiratory
- probability calibration and threshold policy
- SHAP values stored by prediction_id
- health score derived from calibrated risk + lifestyle/vitals

|
|
|
|
v

Explanation/Chat Context Builder

- latest prediction and SHAP
- labs/vitals/profile/history
- red-flag safety checks
- personalization policy

|
v

Safe Generation Layer <----- Evidence Bundle

- context-only generation
- JSON schema enforcement
- source citations
- unsupported-claim checker
- emergency fallback
- audit logs

|
v

API/UI: insights, preventive recommendations, chat, notifications, timeline

Curated Medical Knowledge

guidelines, patient education,
lab interpretation notes,
internal approved content,
source/version metadata

|
v

Hybrid RAG Retrieval

Qdrant dense BGE-M3 vectors
+ sparse BM25/SPLADE/BGE-M3
+ RRF fusion + reranker

|
v

Recommended ML Architecture

- Replace the synthetic RandomForest production role with calibrated LightGBM or XGBoost after real labels exist.
- Keep RandomForest as a regression-test baseline and interpretability fallback.

- Use disease-specific heads instead of one undifferentiated risk score.
- Use a shared feature snapshot table as the online feature source, but add a training-grade offline feature dataset with immutable feature versioning.
- Add explicit missingness indicators because missing health data is clinically informative and current zero-imputation can distort risk.
- Calibrate each head with held-out validation: isotonic regression or Platt scaling, depending on sample size and calibration curve behavior.
- Store model metadata: training data version, label definition, feature schema version, metrics, calibration date, threshold policy, and approval status.

Recommended RAG Architecture

- Create a new Qdrant collection version with named dense and sparse vectors. Do not overwrite the current collection until evaluation passes.
- Use BGE-M3 for dense and sparse retrieval if self-hosting. If using OpenAI embeddings, pair them with BM25/sparse retrieval and a reranker.
- Use section-aware chunks: 150-350 words for patient education, 300-700 words for guideline sections, with title, source, clinical topic, evidence grade, publication date, and audience metadata.
- Run hybrid retrieval: dense and sparse prefetches, RRF fusion, top-20 candidate pool, rerank to top 4-6 evidence passages.
- Use patient context filters carefully. Filter by language/topic/source type, not by risk score alone. Risk score should influence prompt context, not suppress evidence.
- Add a citation validator that rejects generated claims without source support.
- Return no-answer or limited-answer responses when retrieval confidence is low.

Recommended Embedding Model

Default self-hosted recommendation: BAAI/bge-m3. It supports dense, sparse, and multi-vector retrieval modes, 1024-dimensional dense embeddings, and long input length up to 8192 tokens. It is a better strategic fit than BGE-small because ArogyaAI needs hybrid retrieval and likely multilingual expansion.

Fallback low-cost self-hosted recommendation: BAAI/bge-large-en-v1.5 for English-only dense retrieval if BGE-M3 infrastructure is too heavy. It should still be paired with BM25/SPLADE sparse retrieval and a reranker.

Cloud option: OpenAI text-embedding-3-large for best semantic retrieval if PHI and compliance requirements permit; text-embedding-3-small for lower cost. This should be behind a provider abstraction so ArogyaAI can switch embedding providers without changing application logic.

Recommended LLM

Recommended production pattern: a tiered LLM policy. Use deterministic medical safety fallback for red flags, a cost-efficient model for routine education, and a stronger model for high-risk or complex clinical reasoning. Ollama Llama 3.1 8B can remain a local/private fallback, but the production clinical assistant should be backed by a stronger hosted or larger self-hosted model that has passed ArogyaAI's own clinical QA evaluation.

5. Training Strategy

Data To Use

The training dataset should be built from the same sources the product already persists, but with immutable snapshots and leakage controls. Each training row should represent a user-time observation with features calculated only from data available before the prediction timestamp.

Feature Family	Examples	Training Notes
Profile/onboarding	Age, sex, BMI, smoking, alcohol, stress, medications, family history, goals, activity baseline.	Use only stable values valid at observation time. Encode missingness explicitly.
Wearables	Steps, heart rate, sleep duration, sleep efficiency, HRV/RHR proxies, SpO2, variability, slopes, recovery, adherence.	Create 7d/14d/30d aggregates, trend slopes, volatility, and missing-data features.
Labs	Glucose, HbA1c, LDL, HDL, triglycerides, creatinine, TSH, hemoglobin, abnormality flags, lab deltas.	Use units/reference-range normalization. Distinguish patient-measured date from upload date.
Clinical history	Symptoms, severity, duration, onset, negative symptoms, systems involved, possible condition categories.	Convert to structured flags and embeddings only after leakage review.
Timeline/history	Prior reports, prior risk scores, alert history, recent abnormal events, time since last observation.	Avoid using future outcomes or generated recommendations as predictors.
Data quality	Source availability, recency, measurement count, missingness, device connection status.	Missingness is a signal and should be modeled.

Labeling Strategy

- Define explicit disease endpoints. Example: diabetes risk should specify whether it predicts current abnormal glycemic status, 6-month onset, or future physician diagnosis.
- Start with high-confidence weak labels only when clinician labels are unavailable: lab thresholds, repeated abnormal readings, clinician-reviewed reports, or coded diagnosis history.
- Separate diagnostic labels from prognostic labels. A current glucose threshold is not the same task as 12-month diabetes onset.
- Use clinician review to adjudicate ambiguous cases and create a gold validation subset.
- Store `label_source`, `label_date`, `label_window`, `label_confidence`, and exclusion criteria.
- Avoid training the model on its own outputs, recommendations, or RAG-generated explanations.

Feature Engineering Plan

Phase	Feature Work
Baseline parity	Freeze current six features and reproduce RF outputs as a benchmark.
Clinical expansion	Add age, sex, BP, glucose, HbA1c, cholesterol/lipids, smoking/alcohol, family history, disease flags, symptom flags, severity, data completeness.
Temporal expansion	Add 7d/14d/30d means, minimums, maximums, variability, slopes, missingness, recency, and device adherence.
Lab normalization	Normalize units, derive abnormality flags, encode reference-range distance, calculate deltas from previous labs.
Safety constraints	Add monotonic constraints where clinically defensible, for example age and systolic BP for cardiovascular risk, BMI/glucose for metabolic risk.
Production features	Version feature schema; add offline/online consistency tests; prevent leakage with timestamped snapshots.

Evaluation Metrics

Metric	Why It Matters	Minimum Production Gate
ROC-AUC	Measures ranking performance across thresholds.	Report overall and by subgroup. Do not use alone.
PR-AUC	More informative for imbalanced outcomes.	Required for rare disease or high-risk labels.
Sensitivity / recall	Clinical risk products often prefer catching high-risk users.	Set per disease and cost of false negatives.
Specificity	Controls alert fatigue and unnecessary anxiety.	Report at chosen operating thresholds.
F1 / balanced accuracy	Useful summary when classes are imbalanced.	Secondary metric only.
Brier score	Measures probabilistic accuracy.	Required for risk score calibration.
Calibration curve / ECE	Determines whether 0.30 risk means about 30 percent observed risk.	Required before showing risk as probability.
Decision curve / net benefit	Shows whether the model helps at clinically relevant thresholds.	Recommended before automated recommendations.
Subgroup fairness	Checks age, sex, device availability, language, region, and comorbidity strata.	Required before production expansion.

Validation Approach

- Use user-level splits so the same user's timeline does not appear in both train and test.
- Use temporal validation: train on earlier periods and test on later periods to simulate deployment.
- Create a gold clinical validation set reviewed by clinicians for high-risk tasks.
- Use external validation if possible, even if small, before claiming clinical generalization.
- Bootstrap confidence intervals for AUC, PR-AUC, calibration slope/intercept, and threshold metrics.
- Evaluate data missingness strata: users with wearables, users without wearables, users with labs, users with only onboarding.
- Freeze thresholds per disease and document threshold rationale.
- Run silent-mode prospective validation before exposing new risk labels to users.

Model Governance

ArogyaAI should treat every production model as a governed clinical decision-support asset. Each model release should include a model card, intended use, contraindications, training data period, excluded populations, label definition, feature schema, metrics, calibration, subgroup performance, threshold policy, monitoring plan, rollback plan, and human review owner.

6. RAG Improvement Plan

Better Chunking Strategy

- Move from paragraph-only chunking to section-aware chunking that preserves heading hierarchy, source document, clinical topic, evidence type, and audience.
- Use smaller chunks for patient education and larger chunks for guideline sections. Recommended starting range: 150-350 words for concise education; 300-700 words for dense guidelines.
- Store metadata: source_url, source_name, authoring organization, publication_date, review_date, guideline_version, disease_area, evidence_grade, country, language, and intended_audience.
- Use overlap only when needed for long sections. Avoid duplicating entire paragraphs across many chunks because it harms reranking.
- Create specialized lab-reference chunks for units, normal ranges, and interpretation caveats.
- Run chunk-level retrieval evaluation with manually created clinical questions before deployment.

Medical Dataset Sources

The production corpus should be curated, licensed, and versioned. Recommended categories are:

- Government and public-health patient education: NIH/MedlinePlus, CDC, WHO, national public health agencies.
- Clinical guidelines with license review: NICE, ADA, AHA/ACC, ESC, local national guidelines, and specialty society recommendations.
- Internal clinician-approved content: ArogyaAI's own reviewed summaries, lab interpretation policies, escalation criteria, and user-facing safety language.
- Reference data: lab units, biomarker synonyms, common abbreviations, risk thresholds, and device metric definitions.
- Peer-reviewed abstracts only for background retrieval, not direct patient advice, unless reviewed and summarized into approved internal content.

Embedding Upgrade

Step	Action	Success Criterion
1	Keep current BGE-small collection as baseline.	Baseline retrieval metrics and latency recorded.
2	Create new BGE-M3 collection with dense vectors and sparse representations.	Collection supports hybrid query and exact medical term recall.
3	Add BM25/SPLADE or BGE-M3 sparse branch.	Improves recall for labs, drug names, abbreviations, and guideline terms.
4	Add reranker such as bge-reranker-large/base or equivalent.	Top-4 evidence precision improves on clinical QA set.
5	Evaluate OpenAI embeddings only if allowed.	Quality/cost/privacy tradeoff documented.

Hallucination Reduction Techniques

- Use evidence-gated prompts that require citations for every medical claim.
- Add retrieval confidence thresholds. If top evidence is weak, respond with limited answer and ask for more context.
- Run a post-generation grounding checker that maps claims back to source passages.
- Use a strict JSON schema with required sources, safety notes, uncertainty, and follow-up questions.
- Separate diagnosis from education. Never let the model assert disease presence without clinician confirmation.
- Use red-flag deterministic logic before LLM generation for urgent symptoms.
- Log prompt, retrieved document IDs, model version, and final answer hash for auditability.

- Add clinician review workflow for new corpus documents and high-risk answer templates.

RAG Evaluation Plan

Evaluation Type	Dataset	Metric
Retrieval recall	100-300 curated clinical questions mapped to expected source sections.	Recall@5, MRR, nDCG@10.
Answer faithfulness	Generated answers with source passages.	Unsupported-claim rate, citation precision.
Medical safety	Red-flag and ambiguous symptom prompts.	Escalation recall, unsafe reassurance rate.
Personalization	Synthetic patient profiles with labs/vitals.	Correct use of patient context; no fabricated data.
Latency	Representative chat/prediction workloads.	p50, p95, timeout/fallback rate.
Corpus quality	All indexed documents.	Source provenance completeness, review date coverage, duplicate rate.

7. Roadmap

Phase 1: Improve Current Model And Corpus

Workstream	Actions	Exit Criteria
ML baseline	Freeze current RF behavior; create benchmark dataset from real feature snapshots; stop relying on synthetic performance.	RF baseline metrics and calibration reported.
Data quality	Add feature schema inventory, missingness report, unit normalization, timestamp leakage audit.	Feature dataset can be reproduced from a feature_version.
Labels	Define disease-specific targets and create first gold validation subset.	Approved label dictionary and label provenance fields.
RAG corpus	Expand from four Markdown files to reviewed corpus with metadata.	At least 100 curated chunks with source/version metadata.
Evaluation	Build offline ML and RAG evaluation harness.	AUC/PR-AUC/calibration and retrieval recall dashboards exist.
Safety	Document red-flag policy and answer boundaries.	Clinical safety checklist approved.

Phase 2: Introduce Better Model And Hybrid RAG

Workstream	Actions	Exit Criteria
GBDT model	Train LightGBM/XGBoost disease heads with expanded features.	Model beats RF baseline on temporal validation and calibration.
Calibration	Fit isotonic/Platt calibration; choose thresholds per disease.	Calibration slope/intercept, Brier, and threshold metrics approved.
Explainability	Use Tree SHAP; compare top drivers against clinical expectations.	Driver stability and clinician review pass.
Hybrid retrieval	Deploy Qdrant dense+sparse collection, RRF fusion, and reranker.	Retrieval metrics exceed dense-only baseline.
LLM routing	Implement production model policy and fallback behavior.	High-risk queries safely route or fall back deterministically.
Monitoring	Track feature drift, prediction distribution, retrieval failures, LLM fallback rate.	Dashboards and alerts operational.

Phase 3: Scale And Optimize

Workstream	Actions	Exit Criteria
Hybrid temporal ML	Add sequence-derived features or temporal embeddings after longitudinal labels mature.	Temporal model improves net benefit without safety regression.
Feature store	Formal offline/online feature store, backfill jobs, feature contracts, and lineage.	Training-serving skew tests pass.
MLOps	Model registry, staged rollout, canary evaluation, rollback, model cards.	Each model release is traceable and reversible.
RAG governance	Corpus review workflow, automated stale-source detection, citation audits.	Unsupported-claim rate stays below approved threshold.
Clinical QA	Clinician-in-the-loop review for high-impact recommendations.	Escalation and recommendation policies validated.
Performance	Cache hot explanations, pre-index corpus, optimize prompt context, load-test Qdrant and LLM path.	Latency targets met at expected concurrency.

Recommended Timeline

- Weeks 0-4: data audit, label definitions, baseline RF report, corpus expansion, evaluation harness.
- Weeks 5-10: LightGBM/XGBoost training, calibration, SHAP review, hybrid Qdrant prototype, reranker evaluation.

- Weeks 11-16: production governance, staged rollout, monitoring dashboards, LLM routing and safety audits.
- Months 5-8: temporal modeling experiments and external validation if label volume supports it.

8. Risks

Risk	Why It Matters	Mitigation
Data quality	Wearable streams are missing, noisy, device-dependent, and user behavior-dependent.	Model missingness explicitly; validate device source; monitor data completeness and recency.
Label leakage	Reports, generated risk scores, or future labs may leak outcome information into training.	Timestamped feature generation; strict observation windows; leakage tests.
Synthetic bias	Current model behavior reflects synthetic assumptions, not real clinical outcomes.	Use current model only as integration baseline; train on real labeled data.
Overfitting	Small clinical datasets and many derived features can produce unstable models.	Temporal validation, regularization, feature pruning, bootstrapped confidence intervals.
Calibration failure	Users may interpret risk scores as probabilities.	Measure calibration; display ranges/uncertainty; recalibrate by subgroup if needed.
Bias and fairness	Device ownership, language, age, sex, location, and care access can distort model performance.	Subgroup evaluation and mitigation before production claims.
Medical safety	Assistant may under-triage urgent symptoms or overstate diagnoses.	Deterministic red-flag rules, source grounding, no diagnosis language, clinician-reviewed policies.
RAG hallucination	LLM may make claims unsupported by source passages.	Citation validation, retrieval thresholds, no-answer behavior, answer audit logs.
Privacy and compliance	Embeddings and LLM prompts may include PHI.	Provider governance, de-identification, self-hosting where needed, audit logs, access controls.
Operational latency	On-demand indexing, ML refresh, SHAP, retrieval, and LLM calls can stack latency.	Pre-index corpus, cache predictions/explanations, route urgent fallbacks immediately.

Healthcare Safety Position

ArogyaAI should position its intelligence layer as clinical decision support and personal health education, not autonomous diagnosis or treatment. The system should always communicate uncertainty, encourage clinician review for abnormal or persistent patterns, and escalate urgent red flags. The FDA transparency principles and TRIPOD+AI/PROBAST+AI literature both reinforce the need for clear intended use, transparent logic, performance reporting, bias assessment, and human-centered communication.

9. Final Verdict

Best ML model for ArogyaAI: calibrated LightGBM/XGBoost gradient-boosted tabular model.

The production model should use disease-specific heads trained on real labels, not a synthetic general risk formula. LightGBM/XGBoost is the best practical fit because it handles ArogyaAI's current data shape, supports fast inference, works well with missing/heterogeneous structured data, and preserves SHAP explainability. RandomForest should remain only as a baseline and fallback. Neural and temporal models should be explored later after label volume and longitudinal validation mature.

Best RAG architecture: Qdrant hybrid dense+sparse retrieval with BGE-M3, RRF fusion, reranking, and evidence-gated generation.

The RAG layer should keep Qdrant but move beyond dense-only retrieval. BGE-M3 is the recommended self-hosted embedding model because it supports dense, sparse, and multi-vector retrieval and can handle longer, multilingual content. The system needs a curated and versioned medical corpus, reranking, retrieval evaluation, citation validation, and deterministic red-flag safety behavior.

One-Sentence Strategic Answer

ArogyaAI should become a calibrated, explainable tabular risk platform first, then layer stronger temporal modeling on top; in parallel, its RAG layer should become a hybrid, source-governed medical evidence system rather than a small dense-vector demo.

Priority Actions

- Do not present the current synthetic RandomForest as clinically validated.
- Build real disease-specific labels and a temporal validation dataset.
- Train calibrated LightGBM/XGBoost models with richer feature snapshots.
- Expand and govern the RAG corpus before increasing assistant autonomy.
- Upgrade retrieval to Qdrant hybrid dense+sparse with reranking.
- Adopt strict medical safety, citation, and audit controls before production use.

Appendix A. Repository Evidence

Finding	Evidence
Current model type	pipelines/ml_pipeline/train.py imports RandomForestClassifier and trains 250 estimators with max_depth=8 and min_samples_leaf=4.
Current features	pipelines/ml_pipeline/preprocess.py defines FEATURE_NAMES: bmi, hr_mean_7d, steps_avg_7d, sleep_efficiency, lifestyle_score, activity_score.
Artifact metadata	apps/backend/models/diabetes.pkl contains model_version diabetes-rf-20260429180633, label diabetes_risk, synthetic_bootstrap sample_count 1200.
SHAP	pipelines/ml_pipeline/shap_explainer.py uses shap.TreeExplainer and stores feature-level shap_value, direction, and feature_value.
Feature snapshot	pipelines/feature_pipeline/service.py builds profile, vitals, lab, clinical history, disease flags, family history, symptoms, source_breakdown, confidence, and data_availability.
RAG config	pipelines/rag_pipeline/config.py defaults QDRANT_URL http://qdrant:6333, collection medical_knowledge, embedding BAAI/bge-small-en-v1.5, dimensions 384, top_k 4, chunk 300-500 words, Ollama model llama3.1:8b-instruct.
RAG retrieval	pipelines/rag_pipeline/retriever.py creates Qdrant collection with cosine dense vectors and uses client.search with query_vector.
RAG corpus	data/medical_corpus contains four Markdown files: activity_cardiovascular_risk.md, bmi_impact.md, diabetes_risk_factors.md, sleep_metabolic_health.md.
Chat context	apps/backend/services/chat_service.py collects latest ML prediction, health context, RAG context, then calls Ollama/OpenAI-compatible LLM or grounded fallback.
Infrastructure	docker-compose.yml includes backend, frontend, celery-worker, Redis, TimescaleDB/Postgres, Qdrant, prediction-service, rag-service, and wearable-service.

Current Data Inventory

Entity	Production Relevance
UserVital	Canonical wearable time-series values: heart_rate, steps, sleep, SpO2, BP, body temperature, calories.
WearableData	Legacy daily wearable rows retained for step count, sleep duration, sleep score, calories.
LabResult	Structured lab markers per user/report with value, unit, reference_range, category, status, timestamp.
UserProfile	Onboarding/profile fields: demographics, body metrics, family history, medications, sleep, stress, smoking, alcohol, allergies.
ClinicalHistory	Structured symptom intake: complaint, duration, onset, severity, associated and negative symptoms.
FeatureSnapshotRecord	Persisted online feature payload plus hr_mean_7d, steps_avg_7d, sleep_efficiency, BMI, scores, confidence, source_breakdown.
RiskScore	Prediction history with risk_level, overall_score, confidence, model_version, source, risk_payload, health_score, run_id.
ShapValueRecord	Stored feature attribution per prediction for explainability and RAG query building.

Appendix B. Evaluation Checklists

ML Pre-Production Checklist

- Feature schema version frozen and documented.
- Training dataset reproducible from immutable snapshots.
- Labels reviewed for leakage, provenance, and time horizon.
- User-level temporal train/validation/test split created.
- Baseline RF, LightGBM/XGBoost, and simple logistic baseline compared.
- AUC, PR-AUC, sensitivity, specificity, F1, Brier, calibration curve, and subgroup metrics reported.
- Thresholds chosen with documented clinical/product rationale.
- Calibration applied and validated.
- SHAP driver stability reviewed by clinicians.
- Model card, monitoring plan, and rollback plan approved.

RAG Pre-Production Checklist

- Corpus source licenses and review ownership documented.
- Each chunk has source, title, topic, version, publication/review date, and intended audience metadata.
- Hybrid dense+sparse retrieval evaluated against dense-only baseline.
- Reranker improves top-k precision on clinical QA set.
- Unsupported-claim rate measured on generated answers.
- Low-confidence retrieval triggers limited/no-answer response.
- Emergency symptoms trigger deterministic safety response before LLM.
- Prompt, source IDs, model version, and answer hash are logged.
- PHI policy reviewed for embedding and LLM providers.
- Latency and fallback rate pass target thresholds.

Suggested Offline Clinical QA Set

Question Type	Examples
Risk explanation	Why did my diabetes risk increase? Why is BMI a driver? Why did sleep lower my risk?
Lab interpretation	What does HbA1c 6.1 mean? Is LDL 145 concerning? What should I ask my doctor?
Wearable trend	My resting heart rate rose this week. What could explain it?
Symptom safety	I have chest pain and dizziness. What should I do?
Data limitation	I have no recent labs. Can you still estimate diabetes risk?
Contradiction handling	My steps are high but risk is also high. Why?

Appendix C. References

scikit-learn RandomForestClassifier documentation:

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

XGBoost documentation: <https://xgboost.readthedocs.io/en/stable/index.html>

Microsoft Research LightGBM overview: <https://www.microsoft.com/en-us/research/project/lightgbm/>

SHAP TreeExplainer documentation: <https://shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html>

Qdrant Hybrid Queries documentation: <https://qdrant.tech/documentation/concepts/hybrid-queries/>

Qdrant Hybrid Search with FastEmbed tutorial: <https://qdrant.tech/documentation/tutorials-search-engineering/hybrid-search-fastembed/>

BAAI/bge-small-en-v1.5 Hugging Face model card: <https://huggingface.co/BAAI/bge-small-en-v1.5>

BAAI/bge-m3 Hugging Face model card: <https://huggingface.co/BAAI/bge-m3>

BGE-M3 documentation: https://bge-model.com/bge/bge_m3.html

OpenAI embeddings guide: <https://platform.openai.com/docs/guides/embeddings>

OpenAI text-embedding-3-large model page: <https://platform.openai.com/docs/models/text-embedding-3-large>

OpenAI text-embedding-3-small model page: <https://platform.openai.com/docs/models/text-embedding-3-small>

Ollama Llama 3.1 model page: <https://ollama.com/library/llama3.1>

FDA transparency principles for machine learning-enabled medical devices:

<https://www.fda.gov/medical-devices/software-medical-device-samd/transparency-machine-learning-enabled-medical-devices-guiding-principles>

TRIPOD+AI statement, BMJ 2024: <https://www.bmj.com/content/385/bmj-2023-078378>

PROBAST+AI, BMJ 2025: <https://www.bmj.com/content/388/bmj-2024-082505>

End Note

This report intentionally favors production feasibility over theoretical maximum performance. The safest path is to validate the tabular intelligence layer first, expand and govern evidence retrieval, and then introduce deeper temporal modeling only when the data can support it.